

Deployment plan

A runbook for getting Personal Ledger online, updating it safely afterwards, and opening it to a few testers. Work top to bottom — each phase assumes the one before it is done.

The README is the reference doc; this is the ordered checklist.

Phase 0 — Before you start

You already have a Supabase project: the one your local `.env` points at, holding whatever you typed in while building. **Production starts on a new, empty project instead**, and the old one stays exactly where it is as a scratch database.

	Project	Keys live in
Dev	the existing one	<code>.env</code> on this machine — unchanged
Prod	a fresh one, created in Phase 1	Vercel environment variables only

Nothing is copied between them and nothing is deleted. The split is worth the five extra minutes: local experiments cannot reach real users' data, because the local build has never seen the production keys.

- ☐ Count the projects in your Supabase org. **The free tier allows two active projects.** If you're already at two, pause or delete one before Phase 1 — or use the reuse path at the end of Phase 1 instead.
- ☐ Decide the deployed URL (a `*.vercel.app` subdomain is fine to start).
- ☐ Confirm the build passes locally with env vars set: `VITE_SUPABASE_URL=... VITE_SUPABASE_ANON_KEY=... npm run build`
- ☐ Confirm `git check-ignore -v .env` reports it as ignored.

Why the build check matters: Vite inlines env vars at build time. Without them, the config guard in `App.jsx` folds to a constant, Rollup treats the whole app as dead code, and you get a bundle containing nothing but the "Not configured" screen. A green build without env vars is not a green build.

Phase 1 — The production database

- ☐ Create a **new** project at `supabase.com` — not the one you've been developing against. Name the two so they can never be confused at a glance: `personal-ledger-prod` and `personal-ledger-dev`. Note the region; pick the one closest to you.
- ☐ Save the database password somewhere durable. Supabase shows it once.
- ☐ **Confirm the SQL editor is open on the new project** before running anything. The project name sits in the top-left switcher; every step below goes to whichever project that switcher names.
- ☐ **SQL Editor** → **New query**. Run the migrations in order, one at a time. None have ever run here, so all four are needed: `- [ ] migrations/001_initial.sql` — tables + row-level security `- [ ]`

`migrations/002_indexes.sql` — indexes on the columns every query sorts by - []

`migrations/003_feedback.sql` — insert-only feedback table - []

`migrations/004_loan_terms.sql` — loan interest, term, and penalty events

- ☐ Verify RLS is on: **Table Editor** → each of the six tables shows an "RLS enabled" badge. If any table is missing it, stop and re-run `001`.
- ☐ Prove the database is actually empty:

```
```sql
select (select count(*) from transactions) as transactions,
 (select count(*) from loans) as loans,
 (select count(*) from loan_events) as loan_events,
 (select count(*) from budgets) as budgets,
 (select count(*) from auth.users) as users;
```
```

Every column must read ``0``. A non-zero anywhere means the switcher is on the old project — stop and check before going further.

- ☐ **Project Settings** → **API**. Copy the **Project URL** and the **anon public** key. Never copy the `service_role` key into this app.
- ☐ **Do not paste these into `.env`**. They go into Vercel in Phase 3 and nowhere else. Leaving `.env` pointed at dev is the whole mechanism that keeps the two apart.

Sanity check

Re-run every migration file a second time. They must all succeed with no errors and no data change. If any of them errors, the file isn't idempotent and must be fixed before it ever runs against real data.

Alternative: reusing the existing project

Only if you're at the free tier's two-project cap and don't want to give one up. This **destroys the data in your dev project** and cannot be undone — there are no backups on the free tier.

- ☐ Run `migrations/004_loan_terms.sql`. It's the one file this project has never seen; `001–003` already ran here while you were building.
- ☐ Empty the tables. The schema is already correct, so deleting rows is all that's needed — nothing gets recreated:

```
```sql
-- Order matters: loan_events references loans.
delete from loan_events where true;
delete from loans where true;
delete from transactions where true;
delete from budgets where true;
```

```
delete from categories where true;
delete from feedback where true;
...
```

- ☐ Re-run the count query from Phase 1 and confirm every column reads 0.

Existing sign-ins survive this, and each account re-seeds its default categories on next load. To clear the accounts too, remove them under **Authentication** → **Users** — that cascades and empties every table above anyway.

This stays out of **migrations/** on purpose. Those files run on every fresh database; a **delete** among them is one misplaced click from wiping production.

## Phase 2 — Auth configuration

All of this is on the **new** project. The dev project keeps its own auth settings and its own users; the two never see each other.

### Authentication → URL Configuration:

- ☐ Set **Site URL** to your deployed origin.
- ☐ Add to **Redirect URLs**: - **http://localhost:5173/\*\*** — local development - **https://your-app.vercel.app/\*\*** — production

Password reset and email confirmation links both bounce without these. This is the single most common reason a working local build fails in production.

### Authentication → Providers → Email:

- ☐ Email provider enabled (it is by default).
- ☐ Decide on email confirmation. Off is smoother while testing; on is the safer default once real people are using it.

## Phase 3 — First deploy

- ☐ **git add -A && git commit -m "Personal Ledger"** — there are no commits yet. Check **git status** first: **.env** must not appear in what gets staged.
- ☐ Create a GitHub repo and push.
- ☐ Import the repo on **vercel.com**. Framework preset: **Vite**. Build command **npm run build**, output directory **dist**.
- ☐ **Add both environment variables before triggering the first build**, using the **new** project's values from Phase 1 — not the ones in your local **.env**: - **VITE\_SUPABASE\_URL** - **VITE\_SUPABASE\_ANON\_KEY**
- ☐ Deploy.
- ☐ Go back to Phase 2 and replace the placeholder production URL with the real one Vercel assigned.

Smoke test on the deployed URL

The production database has no users yet, so the account you've been testing with locally does not exist here. Sign up again — that's expected, not a bug.

- ☐ Sign up with a real address; confirm the email arrives.
- ☐ Sign in. Add a transaction, a budget, a loan, and a repayment.
- ☐ Check Overview, Analytics, Register, Budgets, Loans, Settings all render.
- ☐ Sign out, use **Forgot password?**, complete the reset, sign in with the new password.
- ☐ Open it on your phone.

---

## Phase 4 — Pushing updates without wiping data

**Code deploys cannot touch your database.** The frontend is a static build: Vercel runs `npm run build` and serves `dist/`. Nothing in that pipeline can reach Postgres. Your data lives entirely in Supabase.

So the normal update loop is just:

```
git add -A && git commit -m "..." && git push
```

Vercel rebuilds and swaps the static files. Data is untouched.

### When a change needs a schema change

Schema changes are a **separate, deliberate act** from deploying code, and they run in the opposite order to what feels natural:

1. Write a new migration file — the next free number in `migrations/`. Never edit one that has already run.
2. Make it additive and idempotent: `create table if not exists, alter table ... add column if not exists, drop policy if exists` before `create policy`.
3. Run it against **dev** first. That's what the dev project is for — a migration that fails there costs you nothing.
4. Run it against **prod** in the SQL editor, still before pushing the code that uses it. The old code ignores a column it doesn't know about; new code against a missing column breaks.
5. Then push the code.

Both projects run the same folder top to bottom, so they never drift. Check the project switcher between steps 3 and 4 — it is the only thing standing between "tested on dev" and "ran on prod by accident".

### The rules that keep data safe

- Never edit a migration that has already been run.
- Every statement idempotent — running the folder top to bottom on a live database must be a no-op that succeeds.
- **No drop table. No truncate. No unqualified delete.**
- Adding a column to a table with rows? Give it a default or allow nulls — `add column x text not null` fails, `... not null default ''` works.

Before anything that looks destructive

```
select count(*) from transactions;
select count(*) from loans;
select count(*) from loan_events;
```

Run it before and after; compare. There are no automatic backups on the free tier — a destructive statement here is unrecoverable.

---

## Phase 5 — Opening it to testers

Sign-up is open, and row-level security scopes every table by `auth.uid()`, so each tester only ever sees their own rows. New accounts seed a starter set of categories on first load. There is nothing to configure to let someone in — just send the URL.

- ☐ Ask a tester to sign up and add a few entries.
- ☐ Confirm from your own account that **you cannot see their data**.
- ☐ Have them send something via the **Feedback** button.
- ☐ Read it in **Table Editor** → **feedback**.
- ☐ Confirm the tester **cannot** read the feedback table from the app.

Two things to handle before inviting several people at once

- **Email is rate-limited.** Supabase's built-in sender allows only a few messages per hour, shared across confirmations and password resets. Fine for testers trickling in; it will block a batch signing up together. To lift it, add custom SMTP (Resend or Brevo) under **Authentication** → **SMTP Settings** — a dashboard change, no code change.
- **Consider turning off email confirmation** during the test window so nobody is stuck waiting on a rate-limited inbox.

Closing sign-ups later

Disable the email provider under **Authentication** → **Providers**, then add people individually with **Authentication** → **Users** → **Invite**.

---

## Phase 6 — Keep it alive

- ☐ The free tier **pauses a project after 7 days with no API traffic**. Data isn't deleted, but the app goes offline until you resume it manually. Only prod needs the keep-alive; letting dev pause between sessions is fine, and resuming it takes a click. Point a free [UptimeRobot](#) HTTP monitor at your Supabase project URL on a few-day interval, or run a scheduled GitHub Action that `curls` it.
  - ☐ Skim **Table Editor** → **feedback** now and then; nothing notifies you.
- 

## Known limits

Worth knowing before they surprise you:

- **All data loads at once.** Every transaction is fetched on login and filtered in the browser. Fine for years of personal use; if it ever slows down, move the month filter server-side with `.gte('date', ...).lte('date', ...)` in `useLedgerData.js`.
- **Categories are denormalised text**, with no foreign key to the `categories` table. Renaming a category won't move existing rows onto the new name.
- **The recurring-charge detector stores nothing.** It re-derives from loaded transactions each time, so there's no state to migrate — but also no way to dismiss a false positive yet.
- **Two npm advisories** (`esbuild` via `vite`) affect the dev server only, not the static production build. The fix is a breaking Vite 8 upgrade; not worth taking right now.